



USC University of
Southern California

From ReAct to LangGraph

Planning Patterns

CSCI 599 — Agentic AI · Lecture 6 / Week 3

Prof. Marco Papa



Where we left off

The previous lecture closed with — "ReAct is the loop; planning patterns are what you do when the loop alone isn't enough"

Today we name the patterns, see them in code, see them fail

- Self-consistency, Tree-of-Thought, Plan-and-Execute, Reflexion

And see where they live in production — LangGraph

Closing — a worked example composing three patterns together

When ReAct Isn't Enough



Three Task Shapes Bare ReAct Handles Poorly

Task lacks structure — ReAct decides as it goes, no advance plan

- Long-horizon tasks drift; intermediate steps lose the goal

Single reasoning path unreliable — one path can be wrong

- Harder problems can benefit from sampling or search

Failures repeat across runs — ReAct has no memory between sessions

- Same agent, same task, same mistake, every time

Pattern map — four shapes, four patterns

Task shape	Pattern	What it adds
Single path unreliable	Self-consistency	Sample N paths, vote on answer
Single path + branching	Tree-of-Thought	Sample, evaluate, prune, search
Lacks advance structure	Plan-and-Execute	Plan first, execute, re-plan on failure
Failures repeat across runs	Reflexion	Verbal self-reflection, persisted as memory

ReAct can be an executor inside these patterns; it is not required by all of them.

Sampling Reasoning Paths



Self-consistency — the technique we held back

Wang et al. 2022 — arXiv:2203.11171

Sample N separate reasoning paths from the same prompt

- Diversity from temperature (chat family) or internal sampling (reasoning family)

Take the majority-vote answer across the N paths

- Diversity in reasoning, consensus in answer

Held back from the previous lecture because it belongs next to ToT and Plan-and-Execute — it's a sampling/decoding strategy, not a prompting technique

Self-consistency in code

```
from collections import Counter

def self_consistency(question: str, n: int = 10) -> str:
    answers = []
    for _ in range(n):
        r = client.chat.completions.create(
            model=MODEL,
            messages=[
                {"role": "system",
                 "content": "Think step by step. "
                             "End: ANSWER=N (a plain number)"},
                {"role": "user", "content": question},
            ],
        )
        answers.append(
            parse_answer(r.choices[0].message.content))
    return Counter(answers).most_common(1)[0][0]
```

Self-consistency failure modes

Cost scales linearly with N — and $N=10$ isn't always enough

- Cost-vs-accuracy is the operational tradeoff

Distribution-tail problems — correct answer in the 10% tail

- Majority vote is confidently wrong on these problems
- Hard math, adversarial prompts, problems with deceptive features

2026 reality — reasoning-family models expose different diversity controls

- On well-posed problems, repeated samples can collapse to unanimous vote
- When vote diversity is low, agreement is still a useful diagnostic signal

Diagnostic — if the top-2 vote split is close (6 vs 4 of 10), treat the answer as low-agreement and escalate

Tree-of-Thought — branch, evaluate, prune, search

Yao et al. 2023 — arXiv:2305.10601

Self-consistency samples N flat paths in parallel

- Tree-of-Thought samples N branches at each step, evaluates, prunes the bad ones, and continues from the good ones

Four operators — generate thoughts, evaluate states, search (BFS or DFS), prune

More search compute than self-consistency; better on problems where a wrong early step poisons the whole path

ToT mechanics in code

```
def tot_bfs(question: str, depth: int, k: int, b: int) -> str:
    # k = thoughts per step, b = beam width
    frontier = [("", 0.0)]    # (path, score)
    for _ in range(depth):
        candidates = []
        for path, _ in frontier:
            # generate k thoughts continuing this path
            thoughts = generate_thoughts(
                question, path, k=k)
            for t in thoughts:
                # evaluator scores the partial path
                score = evaluate(question, path + t)
                candidates.append((path + t, score))
        # prune - keep the top b
        frontier = sorted(
            candidates, key=lambda x: -x[1])[:b]
    return finalize(frontier[0][0])
```

ToT failure modes

Evaluator quality bounds tree quality

- If the evaluator can't distinguish good from bad partial paths, pruning is random — you've added cost for nothing

Branching factor blow-up

- Even moderate k and b explode at depth — search cost grows rapidly as you increase branching, beam width, and depth

Hardest mode — the evaluator agrees with the model's mistakes

- Same model evaluating itself reinforces shared biases

Self-consistency vs ToT vs ReAct

PAPA'S RULE

Self-consistency for problems where one shot is unreliable but the second shot would help. Tree-of-Thought when the first step matters more than the last. ReAct when the world has to be consulted, not just the model.

Self-consistency adds cost linearly — N times. Easy to start with, easy to budget.

Tree-of-Thought adds cost combinatorially. Reach for it on problems where wrong early steps poison the chain.

ReAct adds tool access. When the bottleneck is information the model doesn't have, sampling won't help.

Planning Ahead, Learning Back



Plan-and-Execute — plan first, execute, re-plan

Wang et al. 2023 — Plan-and-Solve Prompting (arXiv:2305.04091)

- Academic precursor — devise a plan, then execute it step by step

LangGraph operationalizes this as a three-loop pattern

- Planner — one LLM call that produces an ordered task list
- Executor — runs each task (often a ReAct sub-loop)
- Re-planner — fires when execution diverges from plan

Contrast with ReAct — ReAct decides as it goes; Plan-and-Execute commits to a sequence first

Plan-and-Execute in code

```
def plan_and_execute(task: str) -> str:
    plan = planner_llm(task)
    results = []
    step_idx = 0

    while step_idx < len(plan):
        step = plan[step_idx]
        # Execute – often a ReAct sub-loop
        outcome = react_loop(step, context=results)
        results.append(outcome)

        if should_replan(plan, step_idx, outcome):
            plan = replanner_llm(
                task, plan[:step_idx + 1], results)
            step_idx = len(results)
        else:
            step_idx += 1

    return synthesize(task, results)
```

The re-plan trigger — the operational core

```
def should_replan(plan, step_idx, outcome) -> bool:
    # Trigger 1 — executor returned a failure signal
    if outcome.get("status") == "failed":
        return True

    # Trigger 2 — observation contradicts plan assumptions
    if outcome.get("observation_contradicts_plan"):
        return True

    # Trigger 3 — depth limit hit on current step
    if outcome.get("steps_used") >= STEP_BUDGET:
        return True

    return False
```

Each trigger answers a different question:

- 1 — did this step fail?
- 2 — does what we just learned invalidate later steps?
- 3 — are we burning budget without progress?

Reflexion — verbal self-reflection, persisted

Shinn et al. 2023 — arXiv:2303.11366

After each run, the agent reflects on what went wrong in natural language

- "I tried X, it failed because Y, next time I should Z"

The reflection is stored and fed back into subsequent trials

- Memory grows across runs, not within them

Best fit — environments where the same agent runs the same task repeatedly and you want it to learn from failures

The reflection step in code

```
# After the run completes
REFLECTION_PROMPT = """
You just attempted: {task}
Your trace: {trace}
Outcome: {outcome}

Reflect briefly (2-3 sentences).
What went wrong, and what
should a future you do
differently?
"""

reflection = llm(REFLECTION_PROMPT.format(
    task=task, trace=trace,
    outcome=outcome))
```

```
# Memory entry shape
entry = {
    "task_id": task_id,
    "run_idx": run_idx,
    "reflection": reflection,
    "outcome": outcome,
    "ts": now(),
}
memory_store.append(entry)

# Next run loads it back
prior = memory_store.recent(
    task_id, k=3)
system = BASE_SYSTEM + "\n\n" + \
    format_reflections(prior)
```

Reflexion's memory shape



Memory grows monotonically across runs — later we look at how to bound it.

Plan-and-Execute vs Reflexion

Plan-and-Execute is proactive — structure imposed before the run

- Works on long-horizon tasks where the shape is knowable upfront
- Doesn't help if you can't write a sensible plan in advance

Reflexion is reactive — structure emerges from failure

- Works on repeating environments where you have multiple shots
- Doesn't help on single-shot tasks — there's no "next time"

Composition — Plan-and-Execute for structure, Reflexion for cross-run learning

Pattern in Production



Bare orchestration — by hand

```
state = {"task": task, "step": "plan",
        "plan": None, "results": []}

while state["step"] != "done":
    if state["step"] == "plan":
        state["plan"] = planner_llm(state["task"])
        state["step"] = "execute"
    elif state["step"] == "execute":
        for s_ in state["plan"]:
            r = executor(s_, state["results"])
            state["results"].append(r)
            if should_replan(state["plan"], r):
                state["step"] = "plan"
                break
    else:
        state["step"] = "synthesize"
    elif state["step"] == "synthesize":
        state["answer"] = synth(state["results"])
        state["step"] = "done"
```

LangGraph — same orchestration, compiled

```
from langgraph.graph import StateGraph, END
from langgraph.checkpoint.memory import InMemorySaver

graph = StateGraph(AgentState)
graph.add_node("plan", plan_node)
graph.add_node("execute", execute_node)
graph.add_node("synthesize", synthesize_node)

graph.set_entry_point("plan")
graph.add_edge("plan", "execute")
graph.add_conditional_edges(
    "execute",
    lambda s: "plan" if s.needs_replan
              else "synthesize",
)
graph.add_edge("synthesize", END)

checkpointer = InMemorySaver()
app = graph.compile(checkpointer=checkpointer)
config = {"configurable": {"thread_id": "demo-1"}}
result = app.invoke({"task": task}, config)
```

The Supervisor Pattern

```
from langchain.agents import create_agent
from langgraph_supervisor import create_supervisor

research_agent = create_agent(
    model="openai:gpt-4o",
    tools=[web_search],
)
math_agent = create_agent(
    model="openai:gpt-4o",
    tools=[add, multiply],
)

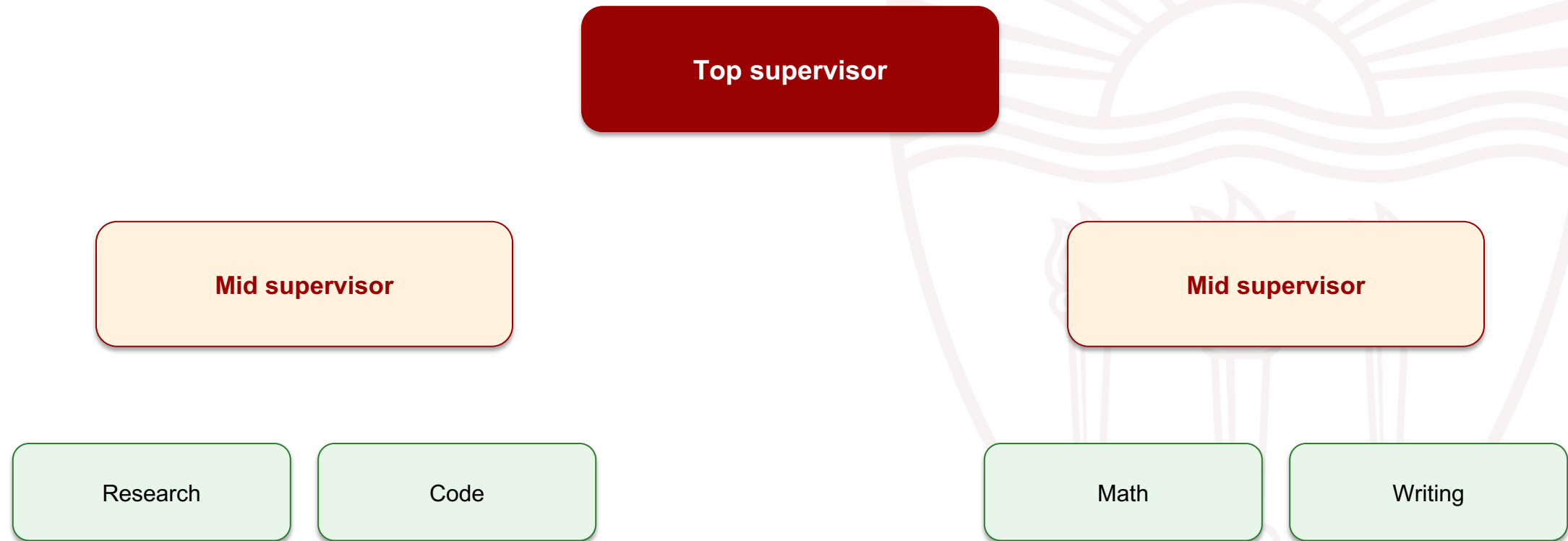
workflow = create_supervisor(
    [research_agent, math_agent],
    model="openai:gpt-4o",
)
app = workflow.compile()
```

2026 guidance:

`create\_agent` is the current agent API.

The supervisor library remains a valid hierarchical option, but current guidance also favors direct tool-calling supervisors for many net-new systems.

Hierarchical — supervisors of supervisors



Use only when team-of-teams structure is real in the domain — not as a default.

Why LangGraph Fits This Production Layer

PAPA'S RULE

LangGraph fits this layer not because it is the only valid choice, but because it makes state, transitions, and orchestration explicit. The operational advantage is that the graph itself becomes the artifact you inspect and manage.

Typed state, checkpointing, replayable debugging

Compiles to a runtime, not just a wrapper around LLM calls

Production deployments at scale — Uber, Klarna, LinkedIn,
Replit

When NOT to use LangGraph

Single-call agents — one retrieval, one generation, no branching

- A function with a try/except is cheaper than a graph

Prototypes you'll throw away in two weeks

- A lightweight role-based framework may be a better fit for a disposable prototype

Single-loop ReAct agents on a fixed tool set

- Bare loop from the previous lecture is the right shape — adding a graph is ceremony

PAPA'S RULE

The graph compiler isn't free — pay the price only when the orchestration is what you're managing.

Composing Patterns



Worked example — multi-document research agent

Task — answer a complex research question from multiple sources

- "Compare how three frameworks handle X — find their docs, compare, and summarize differences"

Requirements that map to patterns:

- Multi-step structure (find, compare, summarize) — Plan-and-Execute
- Branching investigation (which doc to read first, what to follow) — ToT
- This task will be run repeatedly with refreshed sources — Reflexion

Demo boundary — the delivered package implements the four component patterns; this exact three-pattern composition is a worked architecture, not a bundled script

The composition — three patterns wired together

PLAN-AND-EXECUTE (outer loop)

Plan node

Decompose into
3–5 subtasks
[Plan]

Execute node

Per subtask:
ToT branching search
[ToT inside]

Synthesize + reflect

Compose answer,
store reflection
[Reflexion]

Plan-and-Execute provides the outer structure. ToT runs inside the executor for each branching subtask. Reflexion fires at the end and writes to memory for the next run. The delivered demo provides the components and executor seam; this is the worked composition.

When Patterns Fail



Planning failures at multi-pattern scale

Bad plan, no replan trigger fires — executor runs the wrong plan to completion

- Earlier ReAct failure: "looping on a non-issue" — but at the plan level, not the call level

Plan-level context blow-up — every step's results accumulate

- Earlier ReAct failure: "context window blows up" — except now it's across plan steps

Plan hallucinates a tool the executor doesn't have

- Earlier ReAct failure: "hallucinated tools" — promoted from a call problem to a plan problem

Mitigations — the earlier ReAct mitigations apply, plus plan-step truncation

Memory failures — Reflexion's blind spot

Memory grows monotonically — without bounding it grows forever

Stale reflections — environment changed but old reflections persist

- Last year's lesson about a tool that no longer exists

Engineering extensions for bounded memory:

- Recency cap — load only the top-K most recent reflections
- Decay weighting — older reflections weighted lower in the prompt
- Reflection-on-reflections — periodic consolidation pass

Five patterns, five dimensions

Pattern	Cost	Latency	State	When
ReAct	Per-call × N	N × call	Messages list	Tool-using single-loop
Self-consistency	N × per-call	N calls; parallelizable	N traces	Single path unreliable
Tree-of-Thought	Search + evaluation calls	Search depth	Tree of partials	First step matters most
Plan-and-Execute	Plan + steps	Plan + N steps	Plan + results	Long-horizon structure
Reflexion	+1 LLM/run	+ small	Memory store	Repeating environment

These patterns combine — see the worked composition. They are not mutually exclusive choices.

Patterns are tools, not religions

PAPA'S RULE

Pattern names are convenient labels for shapes we've seen work. They are not architectures you should adopt whole — treat them as a vocabulary for design choices, not a list of approved answers.

Composition over allegiance — wire patterns together when the task earns it

Modify the canonical implementation — published code is a starting shape, not a contract

Drop a pattern when its overhead exceeds the problem it solves — and don't be afraid to invent

Next — Tools and Protocols

We've been hand-writing tool schemas; next we plug into shared protocols

MCP — open protocol for connecting AI applications to tools, data, and workflows

A2A — open agent-to-agent interoperability protocol

UCP — Universal Commerce Protocol for agentic commerce

ACP — IBM's Agent Communication Protocol is now part of the A2A ecosystem

LangGraph — orchestration framework; not a protocol

From building loops by hand to plugging into ecosystems